



**PROGETTO
DI INGEGNERIA DEL SOFTWARE**

**“SISTEMA SOFTWARE PER PRENOTAZIONI DI
POSTAZIONI IN BIBLIOTECA
UNIVERSITARIA”**

INDICE

1. SPECIFICHE INFORMALI	1
2. ANALISI E SPECIFICA DEI REQUISITI	3
2.1 ANALISI NOMI-VERBI	3
2.2 REVISIONE DEI REQUISITI	5
2.3 GLOSSARIO DEI TERMINI	6
2.4 CLASSIFICAZIONE DEI REQUISITI	7
2.4.1 Requisiti funzionali.....	7
2.4.2 Requisiti sui dati.....	8
2.4.3 Requisiti Non Funzionali di Qualità.....	9
2.4.4 Vincoli / Altri requisiti	10
2.5 MODELLAZIONE DEI CASI D'USO	11
2.5.1 Attori e casi d'uso	11
2.5.2 Diagramma dei casi d'uso.....	13
2.5.3 Scenari.....	13
2.6 DIAGRAMMA DELLE CLASSI.....	18
2.6.1 Responsabilità	18
2.7 DIAGRAMMI DI SEQUENZA	20
2.7.1 Registrazione	20
2.7.2 Autenticazione.....	20
2.7.3 Effettua Prenotazione	21
2.7.4 Crea Sala Studio.....	24
2.7.5 Elimina Sala Studio	25
2.7.6 Annulla Prenotazione	25
2.7.7 Effettua Check-in	27
2.7.8 VisualizzaProfiloPersonale.....	29
2.7.9 ConsultazioneSaleDisponibili	29
2.8 STATE CHART DIAGRAM	29
2.8.1 statoPrenotazione.....	30
2.8.2 statoPostazione.....	30
2.9 DIAGRAMMA DELLE CLASSI RAFFINATO	30
3. PIANO DI TEST FUNZIONALE.....	31
3.1 REGISTRAZIONE UTENTE.....	31
3.2 AUTENTICAZIONE UTENTE.....	32
3.3 CREAZIONE SALA STUDIO	32
3.4 ELIMINAZIONE SALA STUDIO	32
3.5 EFFETTUAZIONE PRENOTAZIONE.....	33
3.6 ANNULLAMENTO PRENOTAZIONE.....	33
3.7 CHECK-IN POSTAZIONE	33
4. PROGETTAZIONE	34
4.1 DIAGRAMMA DELLE CLASSI.....	34
4.1.1 Traduzione classi ed associazioni.....	34
4.1.2 Pattern BCED	34
4.1.2.1 Package Boundary	34
4.1.2.2 Package Controller	34
4.1.2.3 Package Entity	35
4.1.2.4 Package Database.....	35

4.2 DIAGRAMMI DI SEQUENZA	36
4.2.1 Registrazione	36
5. IMPLEMENTAZIONE	37
5.1 PACKAGE DATABASE	37
5.2 PACKAGE ENTITY	37
5.3 PACKAGE CONTROLLER	37
5.4 PACKAGE BOUNDARY	37
5.5 PACKAGE DTO	37
5.6 DIAGRAMMA DI DEPLOYMENT	38
6. TESTING	39
6.1 TEST STRUTTURALE	39
6.1.1 <i>Complessità ciclomatica</i>	39
6.1.1.1 inserisciAutoModifiche – GestioneParcoAuto	39
6.2 JUNIT – TEST DI UNITÀ	42
6.3 TEST FUNZIONALE	43

1. Specifiche informali

Riportare la Traccia assegnata

Si desidera sviluppare un sistema software per la gestione della prenotazione di postazioni di studio all'interno di una biblioteca universitaria. Il sistema dovrà consentire agli studenti di prenotare una postazione in specifiche fasce orarie, visualizzare le proprie prenotazioni e accedere ai servizi della biblioteca in modo organizzato. Il sistema coinvolge due tipologie di utenti: studenti e bibliotecari, ciascuna con funzionalità e permessi specifici.

Il sistema consente la registrazione di utenti tramite autenticazione con credenziali personali. Al momento della registrazione, ciascun utente deve specificare il proprio ruolo, scegliendo tra studente o bibliotecario, oltre a fornire nome, cognome, email istituzionale e numero di matricola o codice identificativo interno.

Ogni bibliotecario ha la possibilità di creare e gestire una o più sale studio. Per ciascuna sala, mediante apposita interfaccia grafica, è possibile specificare un nome, una descrizione, il numero totale di postazioni disponibili e gli orari di apertura giornalieri. Ogni sala può essere suddivisa opzionalmente in aree differenti, ad esempio area silenziosa, area consultazione o area lavoro di gruppo. Il bibliotecario può inoltre visualizzare in ogni momento l'elenco delle prenotazioni effettuate per ciascuna sala e monitorarne lo stato.

Ogni studente autenticato dispone di una propria interfaccia nella quale può consultare le sale disponibili e verificare, per ciascun giorno, le fasce orarie prenotabili. Una prenotazione viene effettuata selezionando una sala, una data, una fascia oraria e, se prevista, una specifica area o una specifica postazione. Il sistema deve impedire che una stessa postazione venga assegnata contemporaneamente a più studenti nella medesima fascia oraria. Una volta completata la prenotazione, essa entra nello stato "attiva".

Lo studente può visualizzare all'interno del proprio profilo personale l'elenco delle prenotazioni effettuate, con le relative informazioni di sala, data, orario e stato. Deve inoltre essere possibile annullare una prenotazione entro un certo limite temporale precedente all'inizio della fascia prenotata. In caso di annullamento, la postazione torna automaticamente disponibile per altri utenti.

Nel giorno della prenotazione, lo studente può effettuare il check-in tramite una apposita interfaccia grafica, selezionando la prenotazione attiva e confermando la propria presenza. Per semplicità, si può supporre che il check-in debba essere effettuato entro un intervallo di tempo limitato rispetto all'orario di inizio della prenotazione. Se il check-in non viene effettuato entro tale intervallo, la prenotazione passa automaticamente nello stato "scaduta" e la postazione viene resa nuovamente disponibile.

Il bibliotecario, attraverso la propria interfaccia, può consultare in tempo reale la situazione delle sale, visualizzando il numero di postazioni occupate, il numero di postazioni libere e l'elenco delle prenotazioni attive per la giornata corrente. Deve inoltre essere possibile accedere a uno storico delle prenotazioni per ciascuna sala e per ciascuno studente.



Ogni studente ha accesso a un profilo personale che consente di consultare le prenotazioni future, quelle passate, quelle annullate ed eventualmente il numero totale di accessi effettuati alla biblioteca. I bibliotecari, attraverso un'interfaccia dedicata, possono monitorare l'andamento complessivo del servizio, visualizzando il tasso di occupazione delle sale, il numero di prenotazioni giornaliere e il numero di prenotazioni non confermate tramite check-in.

Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. È previsto un sistema di notifiche che avvisi gli studenti della conferma della prenotazione, di eventuali annullamenti e dell'approssimarsi dell'orario prenotato. Il sistema dovrà inoltre garantire la protezione dei dati personali e una corretta gestione dei permessi e delle visibilità tra studenti e bibliotecari.



2. Analisi e specifica dei requisiti

2.1 Analisi nomi-verbi

Il sistema consente la registrazione di utenti tramite autenticazione con credenziali personali. Al momento della registrazione, ciascun utente deve specificare il proprio ruolo, scegliendo tra studente o bibliotecario, oltre a fornire nome, cognome, email istituzionale e numero di matricola o codice identificativo interno.

Ogni bibliotecario ha la possibilità di creare e gestire una o più sale studio. Per ciascuna sala, mediante apposita interfaccia grafica, è possibile specificare un nome, una descrizione, il numero totale di postazioni disponibili e gli orari di apertura giornalieri. Ogni sala può essere suddivisa opzionalmente in aree differenti, ad esempio area silenziosa, area consultazione o area lavoro di gruppo. Il bibliotecario può inoltre visualizzare in ogni momento l'elenco delle prenotazioni effettuate per ciascuna sala e monitorarne lo stato.

Ogni studente autenticato dispone di una propria interfaccia nella quale può consultare le sale disponibili e verificare, per ciascun giorno, le fasce orarie prenotabili. Una prenotazione viene effettuata selezionando una sala, una data, una fascia oraria e, se prevista, una specifica area o una specifica postazione. Il sistema deve impedire che una stessa postazione venga assegnata contemporaneamente a più studenti nella medesima fascia oraria. Una volta completata la prenotazione, essa entra nello stato "attiva".

Lo studente può visualizzare all'interno del proprio profilo personale l'elenco delle prenotazioni effettuate, con le relative informazioni di sala, data, orario e stato. Deve inoltre essere possibile annullare una prenotazione entro un certo limite temporale precedente all'inizio della fascia prenotata. In caso di annullamento, la postazione torna automaticamente disponibile per altri utenti.

Nel giorno della prenotazione, lo studente può effettuare il check-in tramite una apposita interfaccia grafica, selezionando la prenotazione attiva e confermando la propria presenza. Per semplicità, si può supporre che il check-in debba essere effettuato entro un intervallo di tempo limitato rispetto all'orario di inizio della prenotazione. Se il check-in non viene effettuato entro tale intervallo, la prenotazione passa automaticamente nello stato "scaduta" e la postazione viene resa nuovamente disponibile.

Il bibliotecario, attraverso la propria interfaccia, può consultare in tempo reale la situazione delle sale, visualizzando il numero di postazioni occupate, il numero di postazioni libere e l'elenco delle prenotazioni attive per la giornata corrente. Deve inoltre essere possibile accedere a uno storico delle prenotazioni per ciascuna sala e per ciascuno studente.

Ogni studente ha accesso a un profilo personale che consente di consultare le prenotazioni future, quelle passate, quelle annullate ed eventualmente il numero totale di accessi effettuati alla biblioteca. I bibliotecari, attraverso un'interfaccia dedicata, possono monitorare l'andamento complessivo del servizio, visualizzando il tasso di occupazione delle sale, il numero di prenotazioni giornaliere e il numero di prenotazioni non confermate tramite check-in.

Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. È previsto un sistema di notifiche che avvisi gli studenti della conferma



della prenotazione, di eventuali annullamenti e dell'approssimarsi dell'orario prenotato. Il sistema dovrà inoltre garantire la protezione dei dati personali e una corretta gestione dei permessi e delle visibilità tra studenti e bibliotecari.

Note:

- Fascia oraria = orario
- I termini sottolineati indicano i possibili valori dell'attributo tipo della classe Area

LEGENDA:

Classe

Attributo

Funzionalità

Attore

Classe-Attore



2.2 Revisione dei requisiti

1. Il sistema deve consentire la registrazione degli utenti.
2. Il sistema deve offrire all'Utente registrato una funzionalità di accesso tramite autenticazione con credenziali personali.
3. Di ogni Utente si vuole memorizzare nome, cognome, credenziali personali (e-mail istituzionale e password) e ruolo (studente o bibliotecario).
4. Di ogni Studente si vuole memorizzare il numero di matricola.
5. Di ogni Bibliotecario si vuole memorizzare il codice identificativo interno.
6. Il sistema deve offrire al Bibliotecario una funzionalità per creare e gestire Sale Studio.
7. Di ogni Sala Studio si vuole memorizzare nome, descrizione, numero totale di postazioni disponibili e orari di apertura giornalieri.
 - Ambiguità: bisogna capire se le sale sono aperte la domenica e se ci sono giorni di chiusura straordinaria
 - Risposta: aperta nei giorni feriali
8. Il sistema deve consentire al Bibliotecario di suddividere ogni Sala Studio in una o più Aree (tipologia: silenziosa, consultazione, lavoro di gruppo ecc...).
9. Il sistema deve consentire al Bibliotecario di visualizzare l'elenco delle prenotazioni effettuate per ciascuna Sala Studio e monitorarne lo stato.
10. Il sistema deve offrire agli Studenti autenticati una funzionalità per consultare le sale disponibili.
11. Il sistema deve permettere allo Studente di verificare le fasce orarie prenotabili per ogni giorno e per ogni sala.
 - Ambiguità: bisogna capire se per "Disponibilità" si intende lo stato di apertura o chiusura della Sala Studio oppure la presenza di posti disponibili
 - Risposta: presenza di posti disponibili
12. Il sistema deve consentire allo studente di effettuare una Prenotazione selezionando Sala, data e fascia oraria.
 - Ambiguità: bisogna capire se le fasce orarie sono predefinite dal bibliotecario (es. slot fissi di 2h: 8-10, 10-12...) o lo studente le definisce liberamente
 - Risposta: fasce orarie prestabilite dal bibliotecario
13. Il sistema deve consentire di selezionare una specifica Area ed eventualmente una specifica Postazione durante la Prenotazione.
 - Ambiguità: Bisogna capire se le postazioni appartengono a un'area specifica oppure le aree e le postazioni sono due dimensioni indipendenti oppure se quando la sala non è suddivisa in aree, esistono comunque le postazioni numerate; come funziona l'assegnazione della Postazione? Come per i biglietti aerei?
 - Risposta: sì, come i biglietti aerei. C'è sempre almeno un'Area
14. Il sistema deve impedire che una stessa Postazione venga assegnata contemporaneamente a più studenti nella medesima fascia oraria.
15. Il sistema deve aggiornare automaticamente lo stato della Prenotazione ad "attiva" una volta completata.
16. Di ogni Prenotazione si vuole memorizzare Sala Studio, data, fascia oraria e stato.
17. Il sistema deve consentire allo Studente di visualizzare, all'interno del Profilo Personale, le proprie informazioni.
18. Di ogni Profilo Personale si vuole memorizzare Prenotazioni future e passate, Prenotazioni annullate, il numero totale di accessi.
19. Il sistema deve consentire l'annullamento di una Prenotazione entro un limite temporale precedente all'inizio della fascia oraria.



- *Ambiguità: bisogna capire come quantificare l'intervallo di tempo*
 - *Risposta: tolleranza di 6 ore*
20. Il sistema deve rendere la Postazione nuovamente disponibile per altri utenti in caso di annullamento della prenotazione.
21. Il sistema deve offrire allo studente una funzionalità per effettuare il check-in tramite interfaccia grafica.
- *Ambiguità: bisogna capire come avviene tecnicamente il check-in; tramite codice QR generato sulla postazione, geolocalizzazione, o semplice tasto sulla web-app?*
 - *Risposta: tasto di conferma sull'interfaccia*
22. Il sistema deve consentire il check-in solo entro un intervallo di tempo limitato rispetto all'orario di inizio.
23. Il sistema deve aggiornare lo stato della Prenotazione a "scaduta" se in check-in non viene effettuato entro il determinato intervallo.
- *Ambiguità: bisogna capire come quantificare l'intervallo di tempo; cosa succede allo scadere dell'intervallo? Ci sono penalità per lo studente?*
 - *Risposta: tolleranza di 10 minuti. Per le penalità bisogna modellare il comportamento.*
24. Il sistema deve rendere nuovamente disponibile la Postazione se la Prenotazione risulta "scaduta".
25. Il sistema deve offrire al Bibliotecario una funzionalità per consultare, in tempo reale, lo stato delle Sale Studio (postazioni libere/occupate e prenotazioni attive per la giornata corrente).
26. Il sistema deve offrire al Bibliotecario l'accesso allo storico delle Prenotazioni per Sala Studio e Studente.
27. Il sistema deve offrire al Bibliotecario una funzionalità per monitorare il tasso di occupazione, il numero di prenotazioni giornaliere e il numero di prenotazioni non confermate.
28. Il sistema deve essere accessibile via web sia da desktop che da dispositivi mobili con interfaccia grafica intuitiva e responsiva.
29. Il sistema deve inviare le notifiche per confermare le prenotazioni, gli annullamenti e promemoria orari.
- *Ambiguità: Bisogna capire se il sistema di notifiche è interno o esterno e il quando del promemoria*
 - *Risposta: esterno*
30. Il sistema deve garantire la protezione dei dati personali.
31. Il sistema deve garantire la corretta gestione dei permessi di visibilità.

2.3 Glossario dei termini

Termine	Descrizione	Sinonimi
Utente	Persona generica che può registrarsi al sistema, scegliendo il proprio ruolo e può autenticarsi con credenziali personali	
Credenziali	Dati personali (es. e-mail istituzionale e password) utilizzati per l'autenticazione al sistema	



Studente	Utente autenticato con ruolo di studente, identificato da matricola, che può prenotare postazioni nelle sale studio.	
Profilo Personale	Parte di ciascuno Studente in cui consultare lo storico delle prenotazioni e le prenotazioni attive (future)	
Bibliotecario	Utente autenticato con ruolo di bibliotecario, identificato da codice identificativo interno, che gestisce le sale studio e monitora le prenotazioni	
Sala Studio	Ambiente fisico gestito da un bibliotecario, caratterizzato da nome, descrizione, numero di postazioni e orari di apertura	Sala
Area	Suddivisione di una sala studio, caratterizzata da una specifica destinazione d'uso (silenziosa, consultazione, lavoro di gruppo)	
Postazione	Singolo posto a sedere all'interno di una sala studio, appartenente opzionalmente a una specifica area, prenotabile da un solo studente per fascia oraria	
Fascia Oraria	Intervallo di tempo prenotabile all'interno degli orari di apertura di una sala	Orario
Prenotazione	Atto di riservare una postazione in un'area di una sala, per una data e una fascia oraria specifiche, da parte di uno studente	
Storico Prenotazioni	Archivio delle prenotazioni passate (annullate e confermate) consultabile per sala o per studente e numero totale di accessi	
Stato della Prenotazione	Condizione corrente di una prenotazione: può essere "attiva", "annullata", "scaduta" o "confermata"	
Stato della Postazione	Condizione corrente di una postazione: può essere "libera" o "occupata"	
Check-in	Azione con cui lo studente conferma la propria presenza in sala il giorno della prenotazione, entro un intervallo di tempo limitato	
Notifica	Avviso inviato allo studente per confermare prenotazioni, segnalare annullamenti o ricordare l'approssimarsi dell'orario prenotato	

2.4 Classificazione dei requisiti

2.4.1 Requisiti funzionali

ID	Requisito	Origine (n. frase dei requisiti revisionati)
RF01	Il sistema deve consentire la registrazione degli utenti.	1
RF02	Il sistema deve offrire all'Utente registrato una funzionalità di accesso tramite autenticazione con credenziali personali	2



RF03	Il sistema deve offrire al Bibliotecario una funzionalità per creare e gestire Sale Studio	6
RF04	Il sistema deve consentire al Bibliotecario di suddividere ogni Sala Studio in una o più Aree (tipologia: silenziosa, consultazione, lavoro di gruppo etc....).	8
RF05	Il sistema deve consentire al Bibliotecario di visualizzare l'elenco delle prenotazioni effettuate per ciascuna Sala Studio e monitorarne lo stato.	9
RF06	Il sistema deve offrire agli Studenti autenticati una funzionalità per consultare le sale disponibili.	10
RF07	Il sistema deve permettere allo Studente di verificare le fasce orarie prenotabili per ogni giorno e per ogni sala.	11
RF08	Il sistema deve consentire allo studente di effettuare una Prenotazione selezionando Sala, data e fascia oraria	12
RF09	Il sistema deve consentire di selezionare una specifica Area ed eventualmente una Postazione durante la Prenotazione	13
RF10	Il sistema deve consentire allo Studente di visualizzare, all'interno del proprio Profilo Personale, le prenotazioni future, passate, annullate e il numero totale di accessi.	17
RF11	Il sistema deve consentire allo Studente l'annullamento di una Prenotazione entro un limite temporale precedente all'inizio della fascia oraria.	19
RF12	Il sistema deve offrire allo studente una funzionalità per effettuare il check-in tramite interfaccia grafica.	21
RF13	Il sistema deve offrire al Bibliotecario una funzionalità per consultare, in tempo reale, lo stato delle Sale Studio (postazioni libere/occupate e prenotazioni attive per la giornata corrente).	25
RF14	Il sistema deve offrire al Bibliotecario l'accesso allo storico delle Prenotazioni per Sala Studio e Studente.	26
RF15	Il sistema deve offrire al Bibliotecario una funzionalità per monitorare il tasso di occupazione, il numero di prenotazioni giornaliere e il numero di prenotazioni non confermate.	27
RF16	Il sistema deve inviare le notifiche per confermare le prenotazioni, gli annullamenti e promemoria orari.	29

2.4.2 Requisiti sui dati

ID	Requisito	Origine (n. frase dei requisiti revisionati)
RD01	Di ogni Utente si vuole memorizzare nome, cognome, credenziali personali (e-mail istituzionale e password) e ruolo (studente o bibliotecario).	3



RD02	Di ogni Studente si vuole memorizzare il numero di matricola.	4
RD03	Di ogni Bibliotecario si vuole memorizzare il codice identificativo interno.	5
RD04	Di ogni Sala Studio si vuole memorizzare nome, descrizione, numero totale di postazioni disponibili e orari di apertura giornalieri.	7
RD05	Di ogni Prenotazione si vuole memorizzare Sala Studio, data, fascia oraria e stato ed eventualmente l'Area o la Postazione specifica.	16
RD06	Di ogni Profilo Personale si vuole memorizzare Prenotazioni future, passate, annullate ed il numero totale di accessi.	18
RD07	Di ogni Area si vuole memorizzare la tipologia (silenziosa, consultazione, lavoro di gruppo, ...) e la Sala Studio cui appartiene.	8
RD08	Di ogni Postazione si vuole memorizzare la Sala Studio a cui appartiene e se prevista l'Area.	13 e 14
RD09	Di ogni Fascia Oraria si vuole memorizzare l'orario di inizio e l'orario di fine.	11 e 12

2.4.3 Requisiti Non Funzionali di Qualità (si può fare riferimento alle caratteristiche dello Standard ISO 25010)

ID	Categoria	Descrizione	Metrica	Valore atteso	Verifica
RQ-QUAL-01	Sicurezza [Integrità]	Il sistema deve garantire la protezione dei dati personali	% di dati personali accessibili solo previa autenticazione	100%	Test di penetrazione
RQ-QUAL-02	Sicurezza [Riservatezza]	Il sistema deve garantire una corretta gestione dei permessi di visibilità	% di tentativi di accesso a risorse non autorizzate correttamente e bloccati	100%	Test funzionali di accesso cross-ruolo (es. Studente che tenta endpoint da Bibliotecario)
RQ-QUAL-03	Performance [Comportamento temporale]	Il sistema deve offrire al Bibliotecario una funzionalità per consultare,	Tempo di risposta per il caricamento della dashboard delle sale.	≤ 2 secondi nel 95% delle richieste	Test di carico simulando un database pieno di prenotazioni (tutte le postazioni occupate)



		in tempo reale, lo stato delle Sale Studio			
RQ-QUAL-04	Portabilità [Adattabilità]	Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia a grafica intuitiva e responsiva.	% di browser e risoluzioni target su cui l'interfaccia è correttamente fruibile (nessuna funzionalità persa, nessuna sovrapposizione visiva)	100% dei browser principali (Chrome, Firefox, Safari, Edge) e delle risoluzioni rappresentative desktop/tablet/mobile	Test multiplatforma su matrice di browser e dispositivi, con verifica visiva e funzionale a diverse risoluzioni
RQ-QUAL-05	Usabilità [Apprendibilità]	L'interfaccia grafica deve essere intuitiva, permettendo a uno studente di prenotare o fare check-in senza formazione preventiva.	Tempo massimo per completare una prenotazione al primo utilizzo	≤ 3 minuti	Test di usabilità con un campione di studenti non istruiti sul sistema
RQ-QUAL-06	Affidabilità [Tolleranza ai guasti]	Il sistema deve impedire l'assegnazione concorrente di una stessa postazione a più studenti nella medesima fascia oraria.	Numero di "overbooking" o conflitti di prenotazione consentiti dal sistema.	0	Test di concorrenza iniettando richieste simultanee sulla stessa postazione.

2.4.4 Vincoli / Altri requisiti

ID	Requisito	Origine (n. frase dei requisiti revisionati)
----	-----------	--



Vo1	Per l'invio delle notifiche di conferma, annullamento e promemoria orario, deve essere disponibile ed integrato un sistema di messaggistica esterno al sistema.	29
Vo2	Il sistema deve essere sviluppato come applicazione web, senza richiedere l'installazione di software client dedicato nei dispositivi degli utenti.	28
Vo3	Il sistema deve essere conforme alle normative vigenti in materia di privacy (GDPR) per garantire la protezione dei dati personali degli studenti e personale.	30
Vo4	Ogni sala studio deve avere almeno un'area ed ogni area deve essere associata ad almeno una postazione.	13
Vo5	Ogni Fascia Oraria prenotabile deve essere compresa negli orari di apertura giornalieri della Sala Studio cui si riferisce.	12

2.5 Modellazione dei casi d'uso

2.5.1 Attori e casi d'uso

Attori Primari:

- Utente
- Studente
- Bibliotecario
- Tempo

Attori Secondari:

- ServizioNotifiche

Casi d'uso:

- **UC1:** Registrazione
- **UC2:** Autenticazione
- **UC3:** CreaSalaStudio
- **UC4:** EliminaSalaStudio
- **UC5:** MonitoraPrenotazioni
- **UC6:** ConsultazioneSaleDisponibili
- **UC7:** EffettuaPrenotazione
- **UC8:** VisualizzaProfiloPersonale
- **UC9:** AnnullaPrenotazione
- **UC10:** EffettuaCheck-in
- **UC11:** MonitoraSale
- **UC12:** ConsultaStoricoPrenotazioni
- **UC13:** MonitoraStatisticheServizio
- **UC14:** InvioPromemoria

Casi d'uso di inclusione:

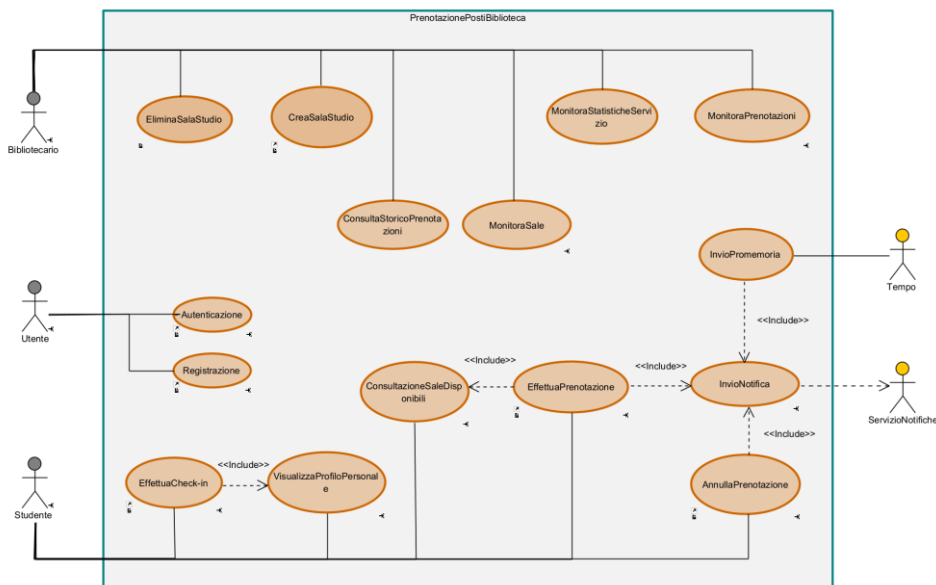


- **UC15:** InvioNotifica

Caso d'uso	Attori Primari	Attori Secondari	Incl. / Ext.	Requisiti corrispondenti
UC1: Registrazione	Utente	-	-	RF01
UC2: Autenticazione	Utente	-	-	RF02
UC3: CreaSalaStudio	Bibliotecario	-	-	RF03, RF04
UC4: EliminaSalaStudio	Bibliotecario	-	-	RF03
UC5: MonitoraPrenotazioni	Bibliotecario	-	-	RF05
UC6: ConsultazioneSaleDisponibili	Studiante	-	Incluso in EffettuaPrenotazione	RF06, RF07
UC7: EffettuaPrenotazione	Studiante	-	Include ConsultaSaleStudio, InvioNotifica	RF08, RF09
UC8: VisualizzaProfiloPersonale	Studiante	-	-	RF10
UC9: AnnullaPrenotazione	Studiante	-	Include InvioNotifica	RF11
UC10: EffettuaCheck-in	Studiante	-	-	RF12
UC11: MonitoraSale	Bibliotecario	-	-	RF13
UC12: ConsultaStoricoPrenotazioni	Bibliotecario	-	-	RF14
UC13: MonitoraStatisticheServizio	Bibliotecario	-	-	RF15
UC14: InvioNotifica	-	ServizioNotifiche	Incluso in EffettuaPrenotazione, AnnullaPrenotazione, InvioPromemoria	RF16
UC15: InvioPromemoria	Tempo	ServizioNotifiche	Include InvioNotifica	RF16



2.5.2 Diagramma dei casi d'uso



2.5.3 Scenari

Caso d'uso:	Registrazione
Attore primario	Utente
Attore secondario	-
Descrizione	Un Utente non registrato richiede di registrarsi nel sistema delle prenotazioni delle postazioni della biblioteca.
Pre-Condizioni	L'Utente non registrato con e-mail istituzionale.
Sequenza di eventi principale	1. Il caso d'uso inizia quando l'Utente accede alla sezione dedicata alla registrazione dall'interfaccia web. 2. L'Utente specifica ruolo, nome, cognome, e-mail istituzionale e sceglie una password. 3. Il sistema verifica l'esistenza del ruolo selezionato. 4. if Il ruolo selezionato è "Studente": 4.1. L'Utente inserisce il numero di matricola. 5. else if Il ruolo selezionato è "Bibliotecario": 5.1. L'Utente inserisce il codice identificativo interno. end if 6. Il sistema verifica la correttezza dei dati e l'univocità dell'account. 7. Il sistema crea il nuovo profilo utente e memorizza i dati.
Post-Condizioni	L'Utente viene registrato nel sistema e può autenticarsi tramite le proprie credenziali.



Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 6, se l'e-mail istituzionale o il codice/matricola risultano già presenti nel sistema il sistema restituisce un messaggio di errore e invita l'utente a recuperare le credenziali o correggere i dati. Al punto 7, se i formati dei dati non rispettano i vincoli, il sistema segnala l'errore specifico.

Caso d'uso:	Autenticazione
Attore primario	Utente
Attore secondario	-
Descrizione	Un Utente registrato richiede di accedere al sistema delle prenotazioni delle Postazioni della biblioteca tramite le proprie credenziali personali.
Pre-Condizioni	L'Utente deve aver effettuato la <i>Registrazione</i> nel sistema e non deve aver ancora effettuato l'autenticazione.
Sequenza di eventi principale	1. Il caso d'uso inizia quando l'Utente non autenticato richiede di autenticarsi nel sistema delle prenotazioni delle postazioni della biblioteca. 2. L'Utente inserisce la propria e-mail istituzionale e la password scelte in fase di registrazione. 3. Il sistema verifica la correttezza del formato dei dati inseriti. 4. Il sistema verifica la corrispondenza tra e-mail istituzionale e password memorizzate.
Post-Condizioni	L'Utente risulta autenticato nel sistema e può accedere alle funzionalità consentite dal proprio ruolo.
Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 3, se il formato dell'e-mail istituzionale o della password non è valido, il sistema segnala l'errore specifico e invita l'Utente a correggere i dati inseriti. Al punto 4, se l'e-mail istituzionale non risulta presente nel sistema oppure la password non corrisponde, il sistema mostra un messaggio di errore e impedisce l'accesso.

Caso d'uso:	EffettuaPrenotazione
Attore primario	Studente
Attore secondario	ServizioNotifiche
Descrizione	Uno Studente prenota una postazione in una sala studio, specificando data, fascia oraria e area. Lo Studente può scegliere una postazione specificata oppure richiedere l'assegnazione automatica da parte del sistema
Pre-Condizioni	-
Sequenza di eventi principale	1. <<include>> <i>ConsultazioneSaleDisponibili</i>: il sistema mostra le sale studio disponibili con le rispettive aree, postazioni e fasce orarie prenotabili. Lo Studente individua la sala, la data, la fascia oraria e l'area desiderate. 2. Il caso d'uso inizia quando lo Studente, dopo aver individuato sala, data, fascia oraria ed opzionalmente l'area desiderata, avvia la prenotazione. 3. <i>if</i> lo Studente ha indicato un'area specifica: 3.1. Il sistema verifica che nell'area e nella fascia oraria della data selezionate esistano postazioni libere.



	4. else: 4.1. Il sistema verifica che nella sala studio e nella fascia oraria della data selezionate esistano postazioni libere. end if 5. Il sistema mostra le postazioni disponibili. 6. Lo Studente seleziona una postazione specifica oppure richiede l'assegnazione automatica. 7. if lo Studente ha richiesto l'assegnazione automatica: 7.1. Il sistema seleziona la prima postazione libera disponibile nell'area scelta per la specifica fascia oraria. 8. else: 8.1. Il sistema verifica che la postazione selezionata sia ancora disponibile. end if 9. Il sistema registra la prenotazione e ne imposta lo stato ad "attiva". 10. Il sistema aggiorna il profilo personale dello studente 11. <<include>> InvioNotifica: il sistema invia allo Studente una notifica di conferma della prenotazione.
Post-Condizioni	La prenotazione è memorizzata in stato "attiva" ed appare nel profilo personale dello Studente. La postazione assegnata risulta occupata per la fascia oraria e data specificata.
Casi d'uso correlati	<i>ConsultazioneSaleDisponibili, InvioNotifica.</i>
Sequenza di eventi alternativi	Al punto 3.1, se nell'area selezionata non esistono postazioni libere per la fascia oraria scelta, il sistema segnala l'indisponibilità ed il caso d'uso termina. Al punto 4.1, se nella sala studio selezionata non esistono postazioni libere per la fascia oraria scelta, il sistema segnala l'indisponibilità ed il caso d'uso termina. Al punto 8.1, se la postazione selezionata risulta nel frattempo occupata, il sistema segnala l'indisponibilità e ritorna al punto 7 per consentire allo Studente di selezionare una postazione diversa o richiedere l'assegnazione automatica. Al punto 11, se il sistema non riesce a recapitare la notifica di conferma, la prenotazione rimane comunque registrata con stato "attiva" e il sistema segnala allo Studente il mancato invio della notifica. Il caso d'uso termina.

Caso d'uso:	AnnullaPrenotazione
Attore primario	Studente
Attore secondario	ServizioNotifiche
Descrizione	Lo Studente annulla una prenotazione attiva entro il limite temporale consentito prima dell'inizio della fascia prenotata.
Pre-Condizioni	Lo Studente deve possedere almeno una prenotazione attiva.



Sequenza di eventi principale	1. <<include>> VisualizzaProfiloPersonale: il sistema mostra le prenotazioni attive. Lo studente seleziona una prenotazione attiva. 2. Il caso d'uso inizia quando lo Studente chiede di annullare la prenotazione. 3. Il sistema verifica che l'annullamento avvenga almeno 6 ore prima dell'inizio della fascia oraria prenotata. 4. Il sistema imposta allo stato "annullata" la prenotazione selezionata e il sistema rende nuovamente disponibile la postazione. 5. <<include>> InvioNotifica: il sistema invia una notifica di conferma dell'annullamento allo Studente. 6. Il sistema aggiorna il profilo personale dello Studente.
Post-Condizioni	La prenotazione risulta annullata, la postazione torna disponibile e il profilo personale risulta aggiornato.
Casi d'uso correlati	<i>InvioNotifica</i>
Sequenza di eventi alternativi	Al punto 3, se mancano meno di 6 ore all'inizio della fascia oraria prenotata, il sistema impedisce l'annullamento e informa lo Studente che il limite temporale è stato superato. Al punto 5, se il ServizioNotifiche non è disponibile, il sistema mantiene valida l'operazione di annullamento, ma registra il mancato invio della notifica.

Caso d'uso:	CreaSalaStudio
Attore primario	Bibliotecario
Attore secondario	-
Descrizione	Il Bibliotecario crea una nuova sala studio specificando tutte le informazioni necessarie e definendo le relative aree
Pre-Condizioni	-
Sequenza di eventi principale	1. Il caso d'uso inizia quando il Bibliotecario accede alla sezione per la creazione di una nuova sala studio. 2. Il Bibliotecario inserisce nome, descrizione, numero totale di postazioni da creare, orari di apertura giornalieri e numero di aree da creare. 3. Il sistema verifica la validità dei dati inseriti. 4. Il sistema inizializza la nuova Sala Studio con le rispettive postazioni. 5. while ci sono ancora aree da creare 5.1. Il Bibliotecario specifica la tipologia dell'area ed assegna un numero specifico di postazioni all'area. 5.2. Il sistema verifica che l'area contenga almeno una postazione e che il numero di postazioni totale nelle varie aree non superi il limite massimo disponibile per la sala studio. 5.3. Il sistema crea l'area e l'associa alla sala studio corrente. 6. Il sistema memorizza la nuova sala studio con le sue relative aree e postazioni.
Post-Condizioni	La nuova Sala Studio è memorizzata nel sistema con le sue relative aree e postazioni ed è visibile agli studenti.
Casi d'uso correlati	
Sequenza di eventi alternativi	Al punto 3, se i dati inseriti non sono validi o sono incompleti, il sistema mostra un messaggio di errore e richiede la correzione. Al punto 5.2, se la somma delle postazioni delle aree eccede il totale della sala, il sistema deve impedire l'inserimento dell'Area.

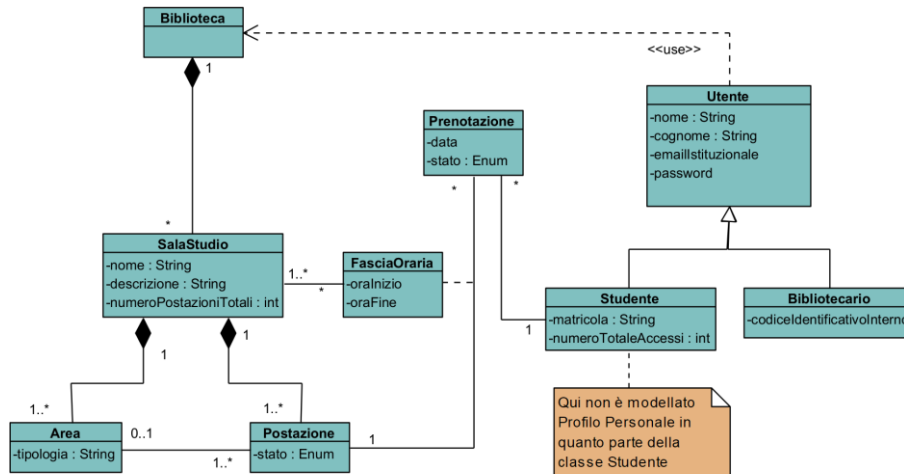


Caso d'uso:	EliminaSalaStudio
Attore primario	Bibliotecario
Attore secondario	-
Descrizione	Il Bibliotecario elimina una sala studio esistente e tutte le relative aree.
Pre-Condizioni	Non devono esserci Prenotazioni attive per la sala scelta.
Sequenza di eventi principale	1. Il caso d'uso inizia quando il Bibliotecario accede alla sezione per l'eliminazione di una sala studio. 2. Il sistema mostra le Sale Studio disponibili. 3. Il Bibliotecario seleziona la sala da eliminare. 4. Il sistema verifica l'esistenza della sala scelta. 5. Il sistema elimina tutte le aree e postazioni associate alla Sala Studio 6. Il sistema elimina la Sala Studio.
Post-Condizioni	L'area non è più presente nel sistema e non è più visibile agli studenti
Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 4, se la sala selezionata non esiste, il sistema segnala l'incongruenza ed invita a scegliere nuovamente una sala.

Caso d'uso:	EffettuaCheck-in
Attore primario	Studente
Attore secondario	-
Descrizione	Nel giorno della prenotazione, lo Studente conferma la propria presenza effettuando il check-in della Prenotazione attiva tramite interfaccia web.
Pre-Condizioni	Lo studente deve avere una Prenotazione attiva.
Sequenza di eventi principale	1. Il caso d'uso inizia quando lo Studente chiede di fare il check-in di una prenotazione attiva nella giornata corrente. 2. Il sistema verifica che la prenotazione abbia scadenza nella giornata corrente. 3. Il sistema verifica che l'operazione sia avvenuta all'interno dell'intervallo prestabilito. 4. Il sistema aggiorna lo stato della Prenotazione a "confermata".
Post-Condizioni	La prenotazione risulta nello stato "confermata" e la postazione rimane assegnata allo Studente per l'intera fascia oraria.
Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 2, se la prenotazione non ha scadenza nella giornata corrente, il sistema segnala l'incongruenza e il caso d'uso termina. Al punto 3, se il check-in non avviene nell'intervallo prestabilito il sistema imposta la prenotazione come "scaduta" e rende la postazione nuovamente disponibile.



2.6 Diagramma delle classi



2.6.1 Responsabilità

A seguire, una tabella che riassume alcune responsabilità:

RESPONSABILITÀ	CLASSE
<i>Registrazione</i>	Biblioteca
<i>Autenticazione</i>	Biblioteca
<i>CreaSalaStudio</i>	Biblioteca
<i>GestioneArea</i>	SalaStudio
<i>MonitoraPrenotazioni</i>	Biblioteca
<i>ConsultaSaleDisponibili</i>	Biblioteca
<i>EffettuaPrenotazione</i>	Biblioteca
<i>VisualizzareProfiloPersonale</i>	Studente
<i>AnnullaPrenotazione</i>	Prenotazione
<i>EffettuaCheck-in</i>	Prenotazione
<i>MonitoraSale</i>	Biblioteca
<i>MonitoraStatisticheServizio</i>	Biblioteca
<i>ConsultaStoricoPrenotazioni</i>	Studente

- *Registrazione* e *Autenticazione* sono responsabilità di **Biblioteca**, in quanto <<information Expert>> di Utenti.

- *CreaSalaStudio* è responsabilità di **Biblioteca**, in quanto <<creator >> dell'oggetto SalaStudio.
- *GestioneArea* è responsabilità di **Sala Studio**, in quanto <<creator >> dell'oggetto Area.
- *MonitoraPrenotazioni* e *ConsultaSaleDisponibili* sono responsabilità di **Biblioteca**, in quanto <<information expert>> di Prenotazione e Sale Studio.
- *Effettua Prenotazione* è assegnata a **Biblioteca**, poiché agisce come <<creator>> della classe Prenotazione.
- *Annulla Prenotazione* ed *EffettuaCheck-in* sono responsabilità di **Prenotazione**, in quanto <<information expert>> del proprio stato e dei vincoli temporali.
- *ConsultaStoricoPrenotazioni* è responsabilità di **Studente**, in quanto <<information expert>> di Prenotazioni.



2.7 Diagrammi di sequenza

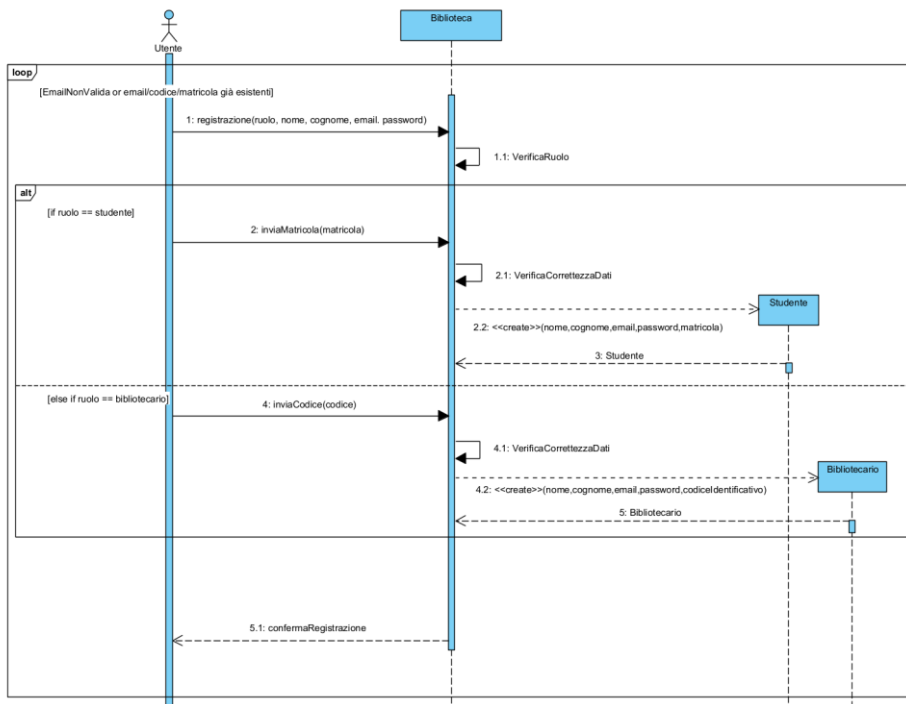
Riportare il diagramma di sequenza di analisi per le funzionalità (ossia i caso d'uso) da implementare (sceglierne una non banale per ogni membro del gruppo) e che saranno sviluppate fino alla codifica in Java ed al test.

Si riportano alcuni esempi di diagrammi di sequenza di analisi per i casi d'uso precedentemente individuati.

2.7.1 Registrazione

La motivazione!!

Commentato [AC1]: [@ANTONIO CACCIATORE](#) qui ti sei dimenticato la motivazione del sequence di Registrazione

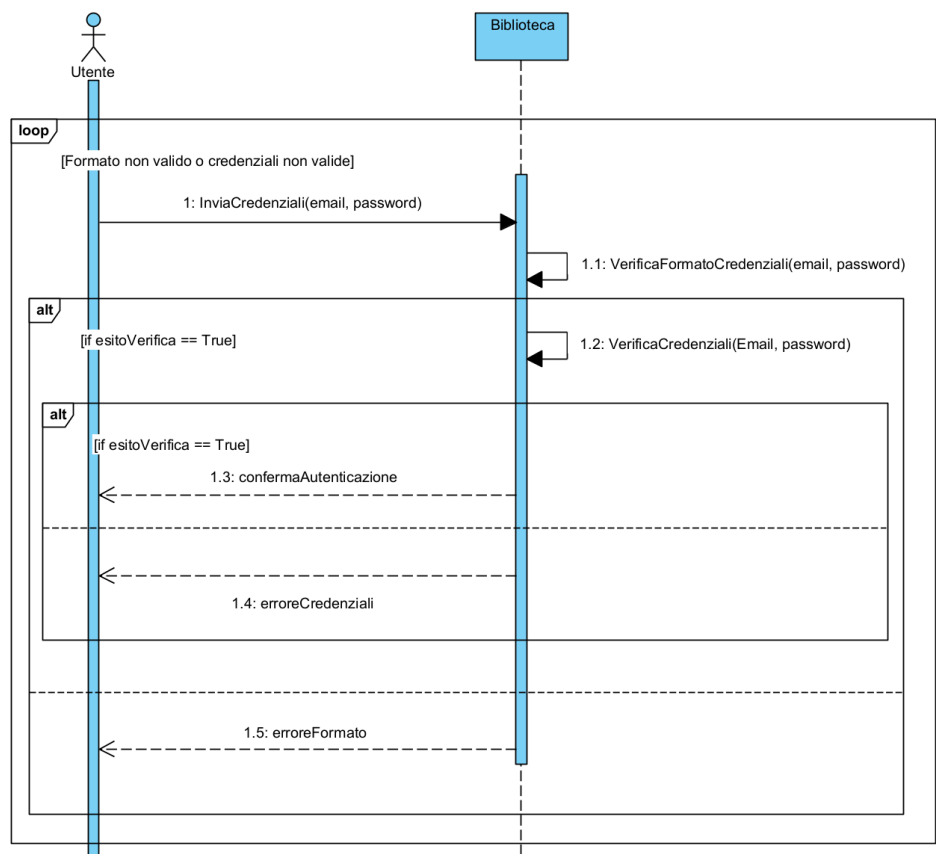


2.7.2 Autenticazione

La motivazione!!

Commentato [AC2]: [@ANTONIO CACCIATORE](#) Qui ti sei dimenticato la motivazione per il sequence di Autenticazione





2.7.3 Effettua Prenotazione

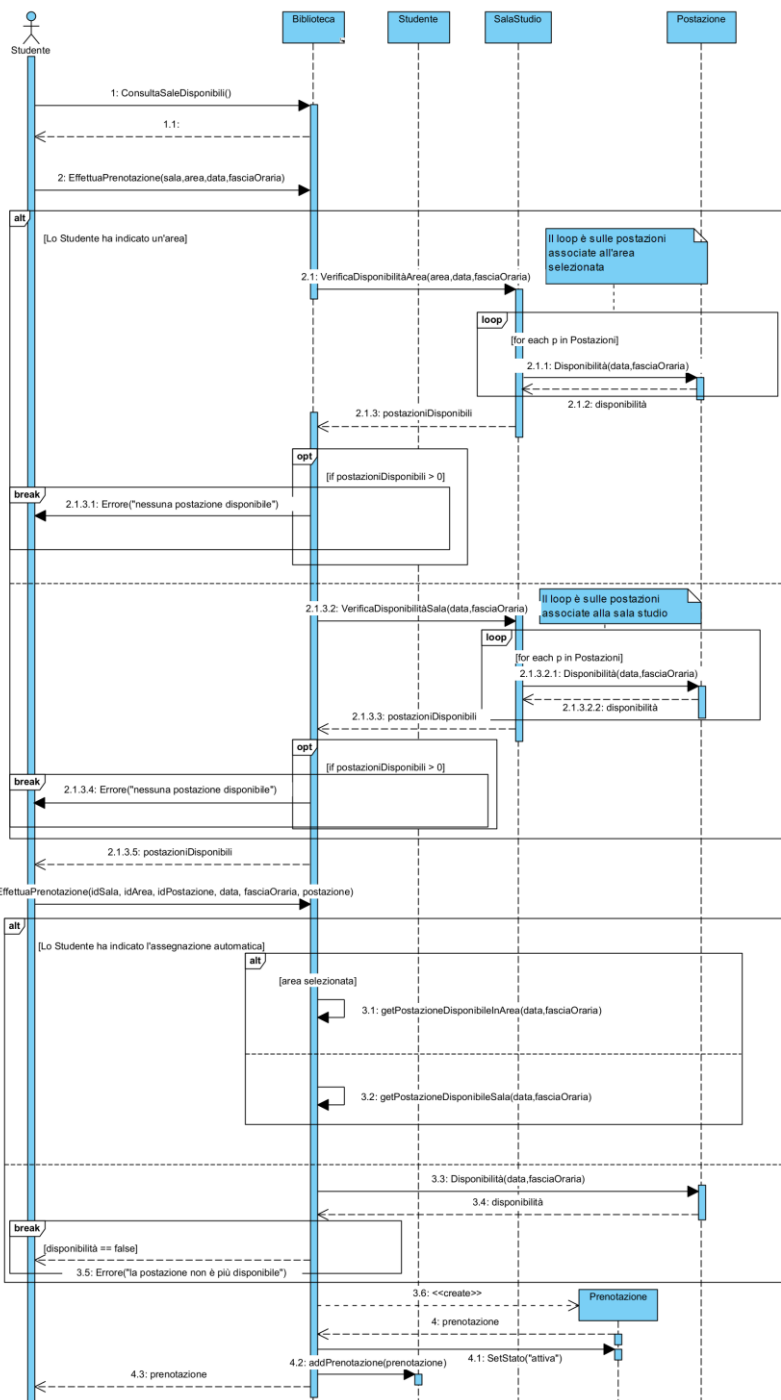
La creazione del suddetto sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso *EffettuaPrenotazione*, ha evidenziato la necessità di definire alcuni metodi:

- Nella classe **Biblioteca** è necessario il metodo **EffettuaPrenotazione(sala, area, data, fasciaOraria)**, che avvia il processo di verifica della disponibilità e la creazione dell'effettiva prenotazione
- Nella classe **SalaStudio** è necessario il metodo **VerificaDisponibilitàArea(area, data, fasciaOraria)**, che itera sulle postazioni della sala studio che sono associate all'area indicata e raccoglie quelle libere per la data e la fascia oraria richieste, restituendo l'insieme delle postazioni disponibili.
- Nella classe **SalaStudio** è necessario il metodo **VerificaDisponibilitàSala(data, fasciaOraria)**, analogo al precedente ma applicato alle sole postazioni contenute nella sala studio, utilizzato quando lo studente non ha indicato alcuna preferenza di area.



- Nella classe **Postazione** è necessario il metodo **Disponibilità(data, fasciaOraria)**, invocato in ogni iterazione del frammento loop per verificare se la singola postazione risulta "libera" nella data e nella fascia oraria indicate.
- Nella classe **Studiante** il metodo **addPrenotazione(prenotazione)**, che aggiunge la prenotazione appena creata alla collezione delle prenotazioni associate allo studente.

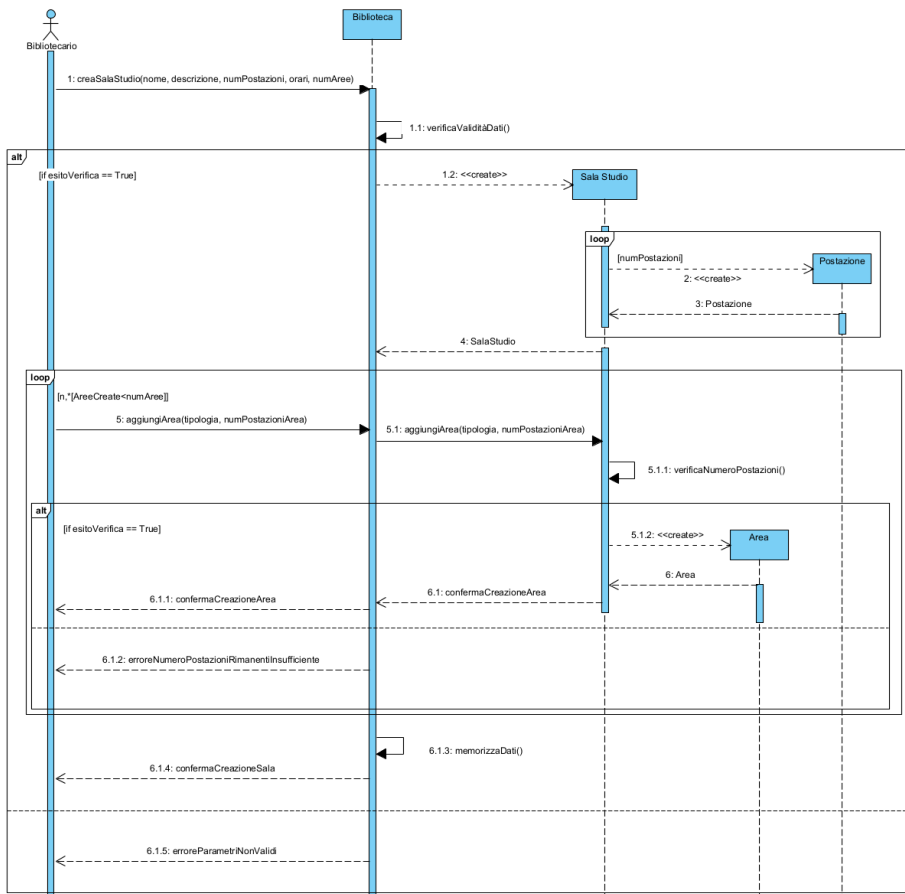




2.7.4 Crea Sala Studio

La creazione del suddetto sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso *CreaSalaStudio*, ha evidenziato la necessità di definire alcuni metodi:

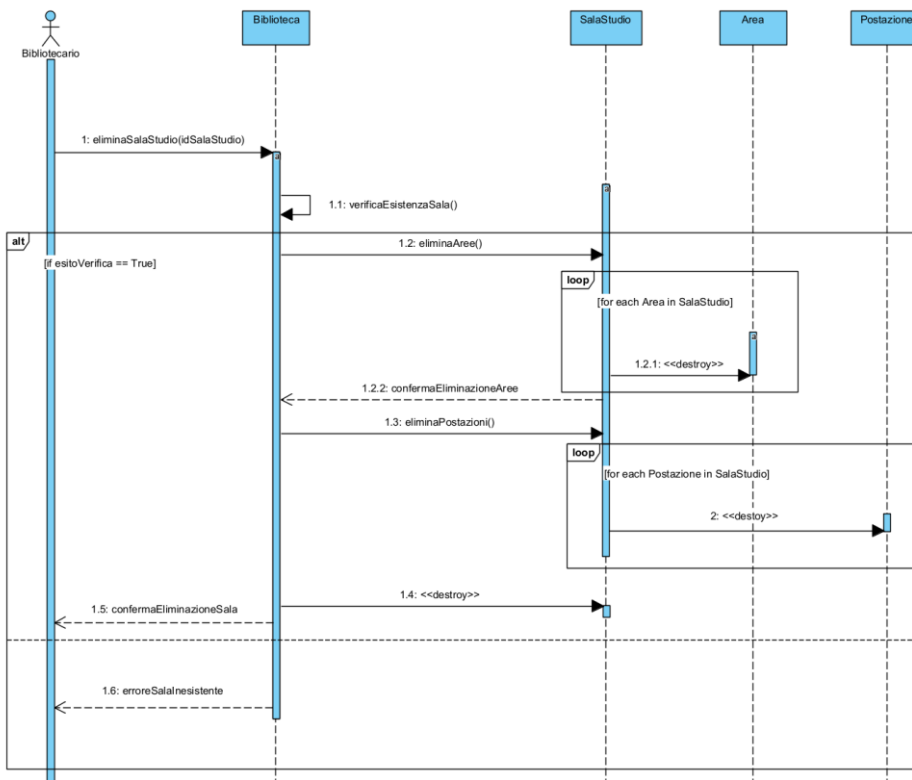
- Nella classe **Biblioteca** il metodo privato **verificaValiditaDati()**, per validare i parametri della sala (nome, orari, capienza, numeroPostazioni, NumeroAree) prima di procedere alla creazione dell'oggetto **SalaStudio**.
- Nella classe **SalaStudio** il metodo **verificaNumeroPostazioni()**. Tale operazione assicura che il numero di postazioni assegnate alle singole aree non ecceda mai la capacità totale definita per la sala stessa.



2.7.5 Elimina Sala Studio

Lo sviluppo del diagramma di sequenza per il caso d'uso *EliminaSalaStudio* ha permesso di individuare tre metodi necessari:

- Il metodo **verificaEsistenzaSala()** all'interno della classe **Biblioteca**. Questo controllo permette al sistema di gestire scenari in cui l'identificativo fornito non corrisponda a una sala esistente, prevenendo errori di esecuzione.
- I metodi **eliminaAree()** ed **eliminaPostazioni()** che, tramite un ciclo iterativo, provvedono alla distruzione (<>) di tutte le rispettive istanze delle classi **Area** e **Postazione** collegate alla sala, prima di procedere alla rimozione definitiva dell'oggetto **SalaStudio** dal sistema.



2.7.6 Annulla Prenotazione

La creazione del seguente sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso **AnnullaPrenotazione**, ha evidenziato la necessità di definire diversi metodi distribuiti tra le classi coinvolte, in modo coerente con le rispettive responsabilità individuate nella sezione 2.6.1.

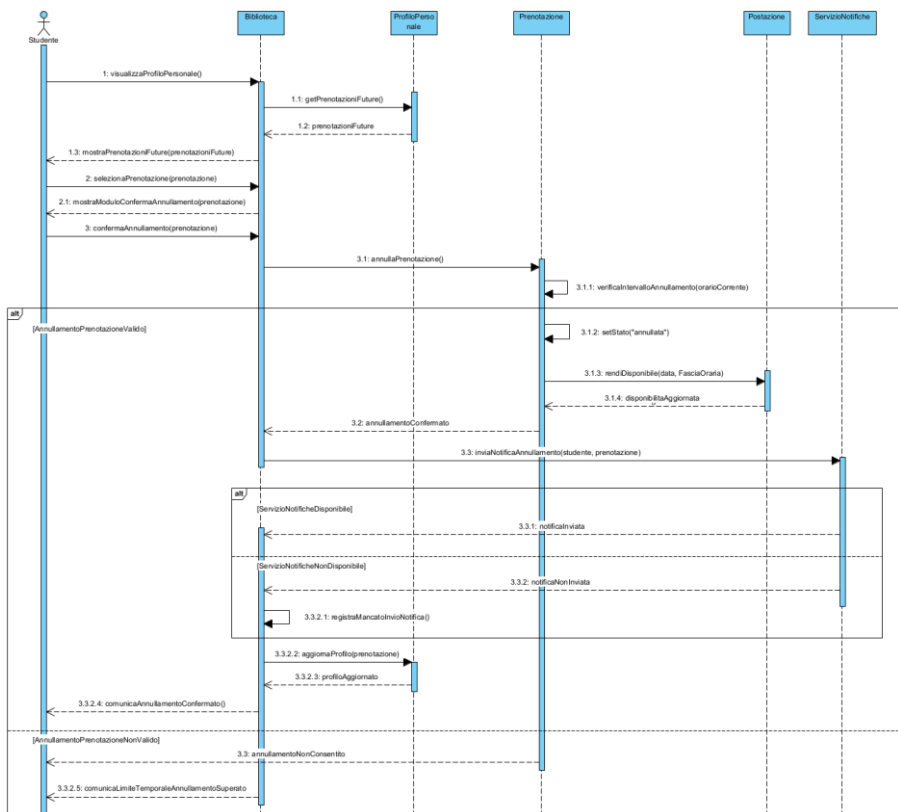
- Nella classe **Biblioteca** è stata introdotta la funzionalità **visualizzaProfiloPersonale()**, invocata dallo **Studente** per accedere al proprio profilo personale. La classe Biblioteca svolge il ruolo di coordinatore dell'interazione,



richiedendo al **ProfiloPersonale** le prenotazioni future dello Studente e mostrandole all'attore.

- Nella classe **ProfiloPersonale** è stato introdotto il metodo **getPrenotazioniFuture()**, che consente di recuperare l'elenco delle prenotazioni future associate allo Studente. Tale metodo è coerente con il principio dell'*Information Expert*, poiché il **ProfiloPersonale** detiene la conoscenza delle prenotazioni dello Studente. In alternativa, per maggiore precisione, il metodo può essere raffinato in **getPrenotazioniFutureAttive()**, così da restituire solo le prenotazioni su cui è effettivamente possibile richiedere l'annullamento.
- Nella classe **Biblioteca** è necessaria la funzionalità **mostraModuloConfermaAnnullamento(prenotazione)**, invocata dopo che lo Studente ha selezionato la prenotazione da annullare. Tale operazione permette allo Studente di confermare esplicitamente l'annullamento prima che il sistema proceda con l'operazione.
- Nella classe **Prenotazione** è stato introdotto il metodo **annullaPrenotazione()**, invocato dalla Biblioteca dopo la conferma dell'operazione da parte dello Studente. Tale metodo rappresenta l'operazione principale del caso d'uso ed è assegnato alla Prenotazione poiché essa conosce il proprio stato, la data e la fascia oraria associata.
- Sempre nella classe **Prenotazione** è stato introdotto il metodo privato **verificaIntervalloAnnullamento(orarioCorrente)**, che consente di verificare se l'annullamento viene richiesto almeno **6 ore prima** dell'inizio della fascia oraria prenotata, come previsto dallo scenario del caso d'uso.
- Nel caso in cui l'annullamento sia consentito, la classe **Prenotazione** invoca il metodo **setStato("annullata")**, aggiornando il proprio stato coerentemente con il comportamento previsto dal sistema.
- Nella classe **Postazione** è stato introdotto il metodo **rendiDisponibile(data, fasciaOraria)**, invocato dalla Prenotazione dopo l'annullamento. Tale metodo consente di rendere nuovamente disponibile la postazione associata alla prenotazione annullata, permettendone l'eventuale assegnazione ad altri studenti.
- Nella classe **Biblioteca** è stata inoltre introdotta la funzionalità **inviaNotificaAnnullamento(studente, prenotazione)**, che consente di richiedere al **ServizioNotifiche** l'invio della notifica di conferma dell'annullamento. Nel caso in cui il ServizioNotifiche non sia disponibile, l'annullamento resta comunque valido, ma viene invocata la funzionalità **registraMancatoInvioNotifica()**, in accordo con lo scenario alternativo previsto dal caso d'uso.
- Infine, nella classe **ProfiloPersonale** è necessario il metodo **aggiornaProfilo(prenotazione)**, che consente di aggiornare l'elenco delle prenotazioni dello Studente dopo l'annullamento. Tale scelta è coerente con il fatto che il **ProfiloPersonale** è *Information Expert* delle prenotazioni associate allo Studente.





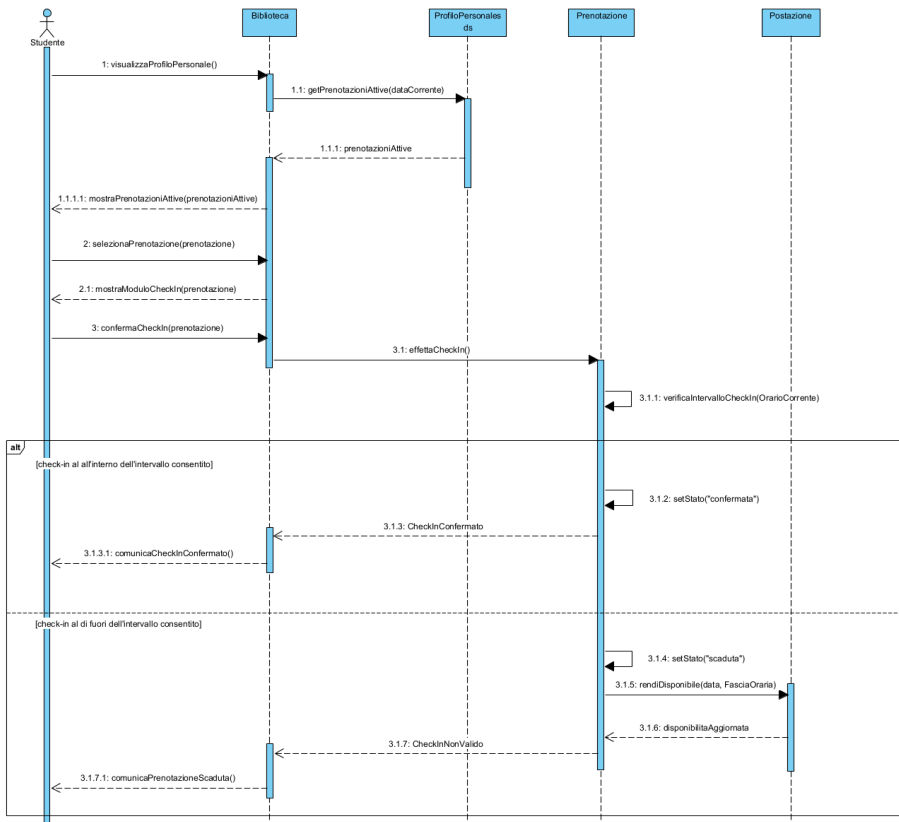
2.7.7 Effettua Check-in

La creazione del seguente sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso **Effettua Check-in**, ha evidenziato la necessità di definire alcuni metodi distribuiti tra le classi coinvolte, in modo coerente con le rispettive responsabilità individuate nella sezione 2.6.1. In particolare, la responsabilità principale del check-in è assegnata alla classe **Prenotazione**, in quanto *Information Expert* del proprio stato e dei vincoli temporali associati al check-in.

- Nella classe **Biblioteca** è stata introdotta la funzionalità **visualizzaProfiloPersonale()**, invocata dallo **Studente** per accedere al proprio profilo e visualizzare le prenotazioni attive della giornata corrente. La classe Biblioteca svolge il ruolo di coordinatore dell'interazione, inoltrando la richiesta alla classe **ProfiloPersonale** e restituendo allo Studente le informazioni ottenute.
- Nella classe **ProfiloPersonale** è stato introdotto il metodo **getPrenotazioniAttive(dataCorrente)**, che consente di recuperare l'elenco delle prenotazioni attive associate allo Studente nella data corrente. Tale metodo è coerente con il principio dell'*Information Expert*, poiché il **ProfiloPersonale** detiene la conoscenza delle prenotazioni dello Studente e permette di selezionare solo quelle su cui è possibile effettuare il check-in.

- Nella classe **Biblioteca** è inoltre necessaria la funzionalità **mostraModuloCheckIn(prenotazione)**, invocata dopo che lo Studente ha selezionato una prenotazione attiva. Tale operazione consente di mostrare allo Studente il modulo attraverso cui confermare la propria presenza.
- Nella classe **Prenotazione** è stato introdotto il metodo **effettuaCheckIn()**, invocato dalla Biblioteca in seguito alla conferma del check-in da parte dello Studente. Tale metodo rappresenta l'operazione principale del caso d'uso ed è stato assegnato alla Prenotazione poiché essa conosce il proprio stato, la data e la fascia oraria a cui è associata.
- Sempre nella classe **Prenotazione** è stato introdotto il metodo privato **verificaIntervalloCheckIn(orarioCorrente)**, che consente di verificare se il check-in viene effettuato entro l'intervallo temporale consentito rispetto all'orario di inizio della prenotazione. In caso positivo, la Prenotazione invoca il metodo **setStato("confermata")**, portando la prenotazione nello stato previsto dallo State Chart Diagram.
- Nel caso in cui il check-in venga effettuato fuori dall'intervallo consentito, la classe **Prenotazione** invoca il metodo **setStato("scaduta")**, aggiornando il proprio stato coerentemente con il comportamento previsto per le prenotazioni non confermate entro i limiti temporali.
- Infine, nella classe **Postazione** è stato introdotto il metodo **rendiDisponibile(data, fasciaOraria)**, invocato dalla Prenotazione nel caso in cui il check-in non sia valido. Tale metodo consente di rendere nuovamente disponibile la postazione associata alla prenotazione scaduta, permettendone l'eventuale assegnazione ad altri studenti.





2.7.8 VisualizzaProfiloPersonale

Motivazione!

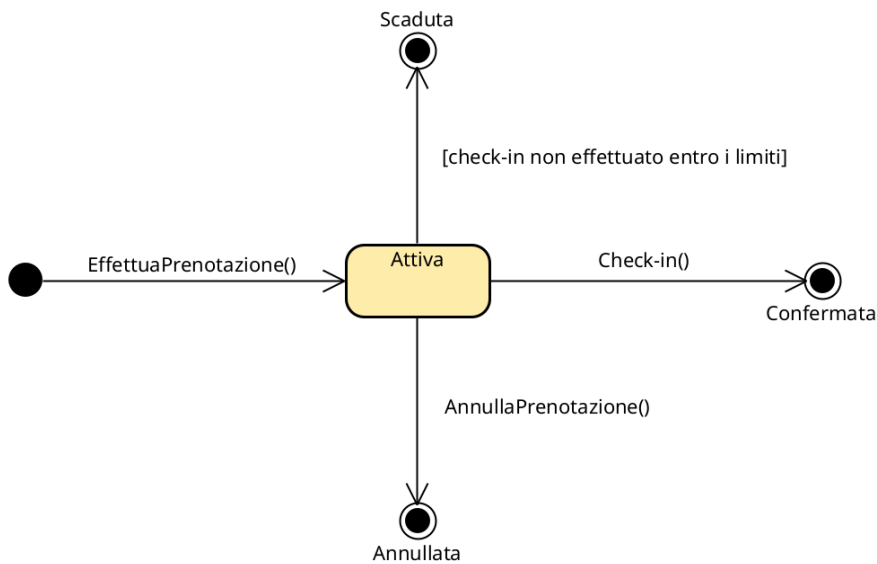
2.7.9 ConsultazioneSaleDisponibili

Motivazione!

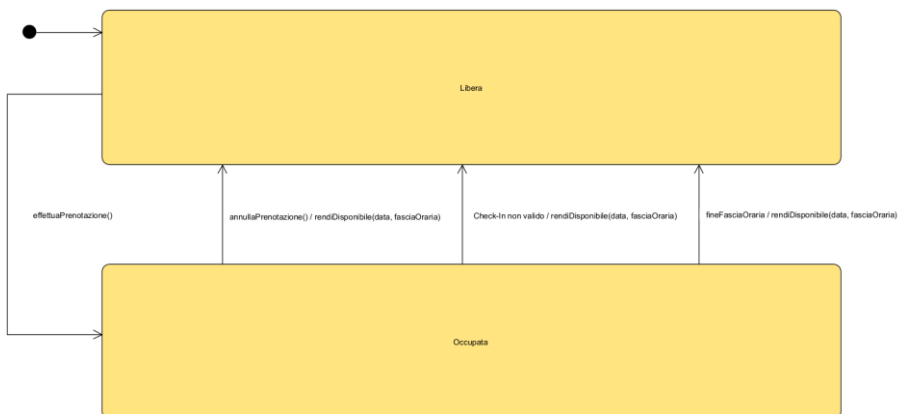
2.8 State Chart Diagram



2.8.1 statoPrenotazione



2.8.2 statoPostazione



2.9 Diagramma delle classi raffinato

Le aggiunte e le modifiche fatte nel corso della costruzione dei Sequence Diagrams hanno determinato lo sviluppo di un [Diagramma delle Classi](#) raffinato che riporta maggiori dettagli sugli attributi e le principali operazioni delle classi:



Riportare CD raffinato

3. Piano di test funzionale

Progettare i casi di test funzionale con la tecnica del *Category Partition Testing*. Descrivere il procedimento di calcolo.

Si intende progettare i casi di test funzionale con la tecnica del *Category Partition Testing*.

3.1 Registrazione Utente

REGISTRAZIONE				
NOME	COGNOME	PATENTE	TIPOPATENTE	CREDITO
- Stringa di caratteri di lunghezza ≤ 40 - Stringa di caratteri di lunghezza > 40 [ERROR] - Stringa che contiene simboli che non sono caratteri [ERROR]	- Stringa di caratteri di lunghezza ≤ 40 - Stringa di caratteri di lunghezza > 40 [ERROR] - Stringa che contiene simboli che non sono caratteri [ERROR]	- Stringa di caratteri alfanumerici di lunghezza $= 10$ - Stringa di caratteri alfanumerici di lunghezza > 10 [ERROR] - Stringa di caratteri alfanumerici di lunghezza < 10 [ERROR] - Stringa contenente caratteri speciali [ERROR]	- Stringa di caratteri alfanumerici di lunghezza ≤ 3 - Stringa di caratteri alfanumerici di lunghezza > 3 [ERROR] - Stringa contenente caratteri speciali [ERROR]	- Numero decimale ≥ 0 - Numero decimale < 0 [ERROR] - Numero che contiene simboli che non sono numeri [ERROR]

Il numero di test da effettuarsi senza particolari vincoli è: $3 * 3 * 4 * 3 * 3 = 324$.

Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è 11 (2 per Nome, 2 per Cognome, 3 per Patente, 2 per TipoPatente, 2 per Credito).

Il numero di test risultante è: $(1*1*1*1*1) + 11 = 12$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi	Nome valido Cognome valido Patente valida TipoPatente valido Credito valido		{Nome: "Mario", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Cliente registrato	Si ricevono le credenziali di accesso per email



3.5 Effettuazione Prenotazione

3.6 Annullamento Prenotazione

3.7 Check-in Postazione



4. Progettazione

4.1 Diagramma delle classi

Riportare il diagramma delle classi di progettazione che rispetti il modello architetturale del BCED (senza violarne le regole di dipendenza fra i livelli). Il class diagram dovrà includere le classi Entity (corrispondenti a quelle di dominio), nonché le Classi Boundary, Controller, e Dati (ossia le classi DAO o Wrapper per l'accesso al Database). Il diagramma dovrà essere organizzato utilizzando i package per raggruppare le classi dello stesso layer.

In questo diagramma saranno inoltre documentate le scelte di progetto fatte, come ad esempio:

- Reificare eventuali classi associative del diagramma delle classi di analisi.
- Specificare argomenti e tipo di ritorno delle operazioni (per quelle più significative, coinvolte nei casi d'uso sviluppati fino alla implementazione).
- Decidere i versi di navigabilità delle associazioni.

4.1.1 Traduzione classi ed associazioni

Spiegare le scelte effettuate

4.1.2 Pattern BCED

4.1.2.1 Package Boundary

Il package Boundary contiene tutti gli oggetti responsabili dell'interfaccia utente e della logica di presentazione; a questo livello tutte le classi corrispondono a delle interfacce e i relativi attributi non sono altro che gli elementi che le compongono, visualizzati a video.

Riportare il Package Boundary

4.1.2.2 Package Controller

Questo package contiene gli oggetti che percepiscono gli eventi generati dalle interazioni con l'interfaccia utente e ne demandano la gestione all'unico componente del sistema software responsabile della gestione della Business Logic, il package Entity.



Riportare il Package Controller

4.1.2.3 Package Entity

Il Package Entity contiene tutti gli oggetti che rappresentano la semantica delle entità del dominio applicativo e corrispondono alle strutture dati presenti all'interno del database di persistenza.

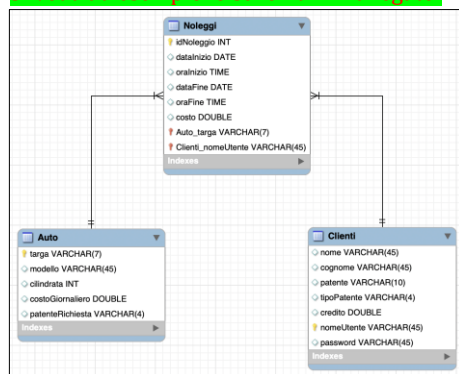
Riportare il Package Entity

4.1.2.4 Package Database

Riportare le scelte progettuali adottate per il database di persistenza

(ossia il Modello E-R costruito ad esempio con l'editor di MySQL Workbench)

Si veda ad esempio lo schema E-R allegato.



Il Package Database contiene tutte le classi responsabili dell'estrazione dei dati dal DB, esponendo una vera e propria interfaccia che di fatto rende indipendenti le classi della Business Logic (Entity) dalla tecnologia di persistenza utilizzata.

In particolare, tra le strategie di risoluzione del problema dell'**impedance mismatch**, che nasce dalla mancata corrispondenza tra il modello Object Oriented e quello relazionale, si è deciso di adottare quella delle classi **DAO** (Data Access Objects), che consiste nell'utilizzo di appositi oggetti per l'accesso ai dati.



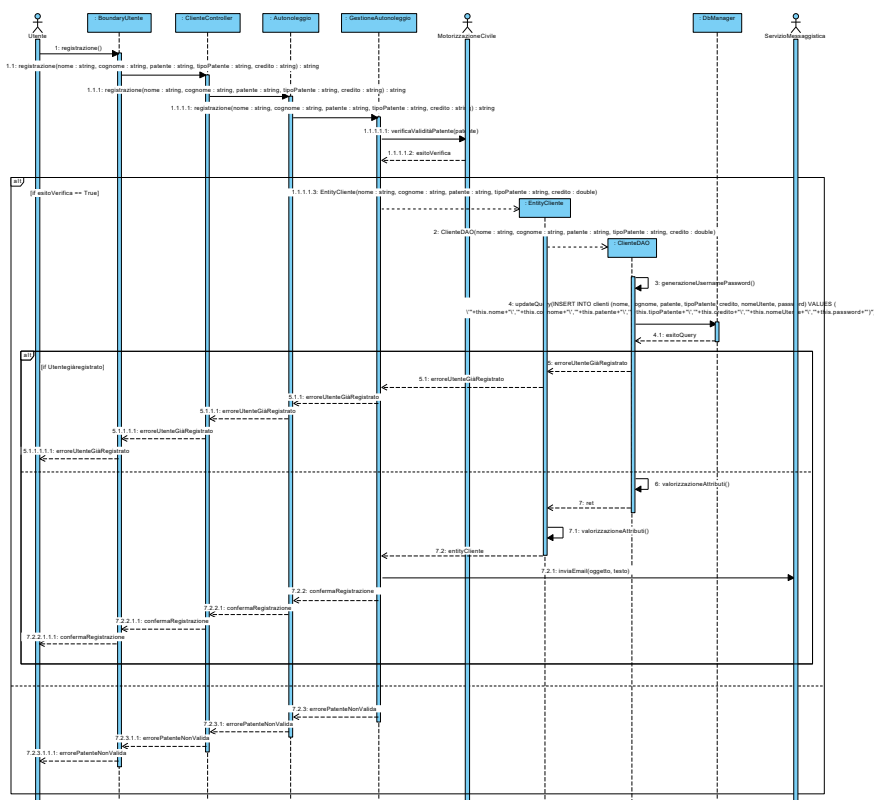
Ognuna di queste classi conterrà i metodi CRUD per l'interrogazione e la manipolazione della corrispondente classe di dominio (*query*), implementati in funzione di un'ulteriore classe, **DBManager**, che costituisce di fatto l'unico punto di accesso vero e proprio al DB, sfruttando i metodi che questa mette a disposizione.

Riportare il Package Database

4.2 Diagrammi di sequenza

Si riportino di seguito i diagrammi di sequenza di progetto per il/i casi d'uso sviluppati fino alla codifica in Java.

4.2.1 Registrazione



I sequence progettati sono stati fondamentali per la corretta implementazione dell'applicazione software ed ha fatto nascere la necessità di definire ulteriori classi, metodi e funzioni, che hanno arricchito passo dopo passo il [Diagramma delle Classi di Progettazione](#), fino ad ottenere la seguente versione finale:

Riportare il Diagramma delle Classi di Progettazione di Dettaglio

5. Implementazione

Non includere il codice sorgente, ma descrivere l'implementazione in Java, descrivendo gli artefatti di codifica:

- 4.1. Elencare:
 - a. package, classi, tipi di eccezione definiti
- 4.2. Elencare gli artefatti necessari per l'installazione ed esecuzione del programma, senza ovviamente l'ambiente di sviluppo come Eclipse (DB h2, eventuali librerie e versioni di Java che l'utilizzatore deve avere installati, file .class, .jar, ...)
- 4.3. Produrre un eventuale diagramma di deployment
- 4.4. Eventualmente inserire la documentazione del codice prodotta con Javadoc (relativamente alle funzionalità implementate)

5.1 Package Database

TBD

5.2 Package Entity

TBD

5.3 Package Controller

TBD

5.4 Package Boundary

TBD

5.5 Package DTO

L'introduzione di tale package, estraneo al pattern BCED, nasce dall'esigenza di mostrare sulla GUI collezioni di elementi.



Da questo punto di vista, il problema principale è proprio quello che, qualora una determinata chiamata a funzione restituisse alla GUI un elenco di entity, questa, per poterlo visualizzare correttamente a video, dovrebbe conoscere di fatto la struttura interna di tale classe Entity, ma ciò porterebbe con sé un accoppiamento troppo elevato e quindi una chiara violazione dei vincoli del pattern a livelli adottato.

Si introduce allora il concetto di **Data Transfer Object (DTO)**, un oggetto in grado di trasportare dati tra processi (nel caso in oggetto tra livelli). Le classi DTO hanno tipicamente una struttura che rispecchia quella dell'entity di cui vanno a supporto, in particolare gli attributi coincidono con quelli dell'entity che si intendono visualizzare a schermo.

5.6 Diagramma di Deployment

I diagrammi di deployment sono utilizzati per mostrare l'architettura fisica del sistema software realizzato; sono particolarmente utili per valutare, durante lo sviluppo, come un'applicazione si distribuisce tra le varie macchine.

- .



6. Testing

6.1 Test Strutturale

Costruire il Control Flow Graph per uno o due dei metodi delle classi implementate (si scelgano metodi non proprio banali), e:

- si mostri il calcolo del numero cicломatico;
 - si indichino i percorsi linearmente indipendenti;
 - si progettino i casi di test per coprire i percorsi individuati.
- Prima o a fianco del CFG riportare il codice Java del metodo.

6.1.1 Complessità ciclomatica

Si intende costruire il Control Flow Graph per due dei metodi delle classi implementate e mostrare il calcolo del numero cicломatico e i percorsi linearmente indipendenti.

6.1.1.1 *inserisciAutoModifiche* – *GestioneParcoAuto*

```
public String inserisciAutoModifiche(String targa, String modello, String cilindrata, String costoGiornaliero, String patenteRichiesta) {
```



```

EntityElencoAuto elenco = new EntityElencoAuto(); (1)
ArrayList<AutoDTO> elencoAuto = elenco.getListaAuto(); (2)
boolean controlloTarga = false; (3)
for(int i=0; i<elencoAuto.size(); i++) { (4)
    if(targa.compareTo(elencoAuto.get(i).getTarga())==0) { (5)
        controlloTarga = true; (6)
    }
}
if(!controlloTarga) { (7)
    return "Errore, l'auto selezionata è inesistente!"; (8)
}
ArrayList<AutoDTO> elencoAutoDeposito = visualizzaElencoAuto(); (9)
controlloTarga = false; (10)
for(int i=0; i<elencoAutoDeposito.size(); i++) { (11)
    if(targa.compareTo(elencoAutoDeposito.get(i).getTarga())==0) { (12)
        controlloTarga = true; (13)
    }
}
if(!controlloTarga) { (14)
    return "Errore, l'auto selezionata non rientra tra quelle in deposito!"; (15)
}
EntityAuto autoDaModificare = new EntityAuto (targa); (16)
return autoDaModificare.modificaAuto(modello, cilindrata, costoGiornaliero,
patenteRichiesta); (17)
}

```

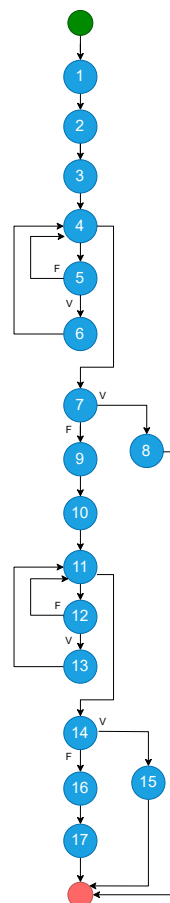
NUMERO CICLOMATICO:

1. numero di regioni chiuse del grafo + 1 = 7
2. numero di nodi predicati (4,5,7,11,12,14) + 1 = 7
3. # archi - # nodi + 2 = (22 - 17) + 2 = 7

CAMMINI:

- 1) 1-2-3-4-7-8
- 2) 1-2-3-4-5-4-7-8
- 3) 1-2-3-4-5-6-4-7-8
- 4) 1-2-3-4-7-9-10-11-14-15
- 5) 1-2-3-4-7-9-10-11-12-11-14-15
- 6) 1-2-3-4-7-9-10-11-12-13-11-14-15
- 7) 1-2-3-4-7-9-10-11-14-16-17

CASI di TEST



TC ID	Cammino Coperto	Descrizione	Condizioni Logiche	Input	Precondizioni	Output atteso	Output effettivo	Esito (PASS/FAIL)
TC_1	1-2-3-4-7-8	Percorso in cui l'auto da modificare non esiste nel sistema	La condizione al punto 7 del CFG è true	targa= CC123EE, modello=PANDA, cilindrata=800, costoGiornaliero=30, patenteRichiesta=B	La targa CC123EE non è memorizzata nel sistema	Errore, l'auto selezionata è inesistente!		PASS/FAIL
2	1-2-3-4-7-9-10-	Percorso in cui l'auto da	La condizione al punto 7	targa= CC123EE, modello=PANDA, cilindrata=800,	La targa CC123EE è memorizzata	Errore, l'auto selezionata	...	PASS/FAIL



	11-14-15	modificare esiste nel sistema, ma non esiste nel deposito	del CFG è false, La condizione al punto 14 del CFG è true	costoGiornaliero=30, patenteRichiesta=B	nel sistema, ma non è presente in deposito	non rientra tra quelle in deposito!		
..								



6.2 JUnit – Test di Unità

Riportare a scopo esemplificativo alcuni casi di utilizzo di **JUnit** per testare alcune classi del progetto, il framework di testing di unità automatizzato per il linguaggio di programmazione Java.



6.3 Test funzionale

Segue una descrizione in forma tabellare dei risultati dell'esecuzione dei test funzionali precedentemente pianificati.

REGISTRAZIONE									
TEST SUITE									
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese	Output ottenuti	Post-condizioni ottenute	Esito (FAIL, PASS)
1	Tutti input validi	Nome valido Cognome valido Patente valida TipoPatente valido Credito valido		{Nome: "Mario", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Cliente registrato	Si ricevono le credenziali di accesso per email	Registrazione avvenuta con successo, controlla la mail per visualizzare le credenziali di accesso	Il Cliente si è registrato e ha ottenuto l'accesso alla piattaforma	PASS
2	Nome stringa > 40 caratteri	Nome stringa > 40 caratteri [ERROR], Cognome, Patente, TipoPatente, Credito validi		{Nome: "Marioooooooooooooooooooooooooooooo ooooooooooooooooooooooooooooooooooooo ooooooooooooooooooooooooooooooooooooo", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Nome troppo lungo!		Errore, il nome inserito è troppo lungo!		PASS
3	Nome stringa con simboli	Nome stringa con simboli [ERROR], Cognome, Patente, TipoPatente, Credito validi		{Nome: "Ma-r.o", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Formato Nome errato!		Errore, il formato del nome inserito non è valido!		PASS
									
12	Credito numero con simboli	Nome, Cognome, Patente, TipoPatente validi, Credito numero con simboli [ERROR]		{Nome: "Mario", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "7@.1!"}	Formato Credito errato!		Errore, il formato del credito inserito non è valido!		PASS

